

Bora Agendar

Documentação do processo e das decisões

Campo	Informação
Integrantes	Gabriel Valenga Silva, Rafael Moura Machado
Disciplina	Desenvolvimento para Dispositivos Móveis
Trabalho	Trabalho 2
Projeto	Hub de agendamento para barbearias, salões e estética automotiva
Repositório	https://github.com/Gabriel-Vs20/Trabalho_2_kotlin

1. Como estava o projeto no Trabalho 1 e o que mudou

No Trabalho 1 o aplicativo tinha três telas e nenhuma navegação de verdade. Para ver uma tela diferente era preciso trocar qual composável estava sendo chamado dentro do `setContent` da `MainActivity`, recompilar e rodar de novo. As telas existiam isoladas:

- Tela de login, com campos de usuário e senha e um botão Entrar cujo `onClick` estava vazio
- Tela de cadastro, com os campos de nome, email, CPF, endereço e data de nascimento
- Tela principal, que era uma montagem estática: a barra verde superior, três quadrados representando estabelecimentos favoritos, um aviso de que não havia horários marcados e três datas fixas

Todo o conteúdo dessa tela principal era texto escrito direto no código. Não existia lista, não existia dado vindo de lugar nenhum e nenhum elemento era clicável.

O estado também era local. Cada tela guardava o que o usuário digitava em `remember { mutableStateOf("") }` dentro do próprio composável, então sair da tela apagava tudo. Havia inclusive um defeito nesse ponto: na tela de cadastro, os campos de CPF, endereço e data de nascimento estavam todos ligados à mesma variável `password`, de forma que digitar em um alterava os três ao mesmo tempo.

O que mudou para chegar até aqui:

Trabalho 1	Trabalho 2
3 telas trocadas na mão no código	10 telas em dois <code>NavHost</code> , navegáveis pela interface
Nenhuma navegação	<code>NavHost</code> central, objeto <code>Rotas</code> , <code>NavigationBar</code> e botão de voltar
Estado solto em cada tela com <code>remember</code>	Estado centralizado em <code>MeuViewModel</code> , compartilhado entre as telas
Nenhuma data class	3 data classes: <code>Estabelecimento</code> , <code>Agendamento</code> e <code>Notificacao</code>
Conteúdo fixo escrito no código	Listas reativas com <code>mutableStateListOf</code> , com adicionar e remover
Nenhum item clicável	Cards abrem telas de detalhe com o item correto

Trabalho 1	Trabalho 2
Botões sem ação	Todo botão executa algo: navega, valida, salva, marca ou remove
Login entrava sem verificar nada	Login só entra se os dados baterem com o cadastro

Tela principal como estava no Trabalho 1, com os quadrados estáticos.

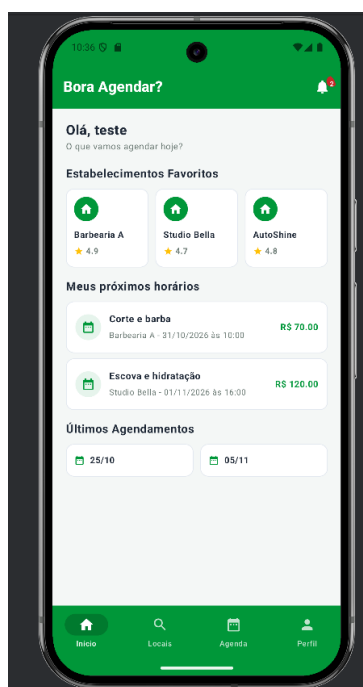
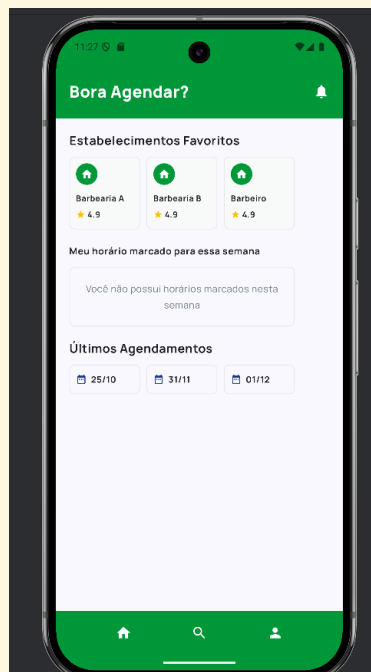


Figura 2. Aba Início do Trabalho 2, com os favoritos, os próximos horários e os últimos agendamentos vindos das listas.

2. Por que essas telas novas e o que cada uma faz

São dez telas. As três primeiras vieram do Trabalho 1 e foram reformuladas, as outras sete são novas.

2.1 Telas que vieram do Trabalho 1

Tela de Login (`TelaLogin.kt`). Continua sendo a porta de entrada, mas agora valida de verdade. O botão Entrar chama `validarLogin()` no `ViewModel`, que confere se existe cadastro e se usuário e senha conferem. Enquanto não bater, a tela mostra o motivo numa faixa vermelha e não navega. O link Cadastre-se já leva para o cadastro.

Tela de Cadastro (`TelaCadastro.kt`). Ganhou um campo de senha, que não existia antes e sem o qual o login não teria o que validar. Recebeu rolagem vertical, porque com seis campos o conteúdo não cabia mais na tela. Cada campo passou a ter seu próprio estado no `ViewModel`, corrigindo o defeito em que três campos dividiam a mesma variável.

Tela Principal (`MinhaTela.kt`). Deixou de ser uma tela com conteúdo próprio e virou o contêiner do aplicativo. Ela monta o `Scaffold` com a `TopAppBar` verde, a `NavigationBar` inferior e um segundo `NavHost` que troca as quatro abas. O conteúdo que antes ficava aqui foi para a aba Início.

2.2 Telas novas

Aba Início (`AbaHome.kt`). É o painel do usuário. Mostra os três primeiros estabelecimentos cadastrados, os agendamentos ainda em aberto e os já concluídos. Diferente do Trabalho 1, nada aqui é fixo: tudo vem das listas do `ViewModel` e todo card abre o detalhe correspondente. Escolhemos manter essa tela porque ela dá uma visão geral sem obrigar o usuário a entrar nas listas.

Aba Locais (`AbaEstabelecimentos.kt`). Primeira tela de lista, com os estabelecimentos. Usa `LazyColumn` e `Card`, tem um formulário que abre e fecha no botão Novo e um ícone de lixeira em cada item. Clicar no card abre o detalhe do estabelecimento. Ela existe porque o usuário precisa de algum lugar para cadastrar onde vai agendar, e não fazia sentido manter os três estabelecimentos chumbados no código.

Aba Agenda (`AbaAgendamentos.kt`). Segunda tela de lista, com os agendamentos. Mesma estrutura da anterior, mais um `Checkbox` que marca o agendamento como concluído e risca o texto. O formulário tem um seletor de estabelecimento em chips com rolagem horizontal, porque um agendamento precisa estar ligado a um local existente. É a tela central do aplicativo, já que agendar é o propósito do projeto.

Aba Perfil (`AbaPerfil.kt`). Mostra os dados do cadastro e permite alterá-los sem precisar refazer a conta. Tem três contadores no topo, calculados a partir das listas: quantidade de locais, de agendamentos e de concluídos. Também tem o botão de sair, que limpa o login e volta para a tela inicial. Incluímos porque, com o login validando contra o cadastro, o usuário precisa de algum lugar para corrigir um email digitado errado.

Detalhe do Estabelecimento (`TelaDetalheEstabelecimento.kt`). Recebe o id pela rota e mostra os dados daquele local, um resumo calculado e a lista de agendamentos feitos ali. É a tela onde colocamos a complexidade extra, explicada na seção 4.

Detalhe do Agendamento (`TelaDetalheAgendamento.kt`). Recebe o id pela rota e permite editar serviço, data, hora, preço e uma observação em campo de múltiplas linhas. Também marca como concluído, exclui o agendamento e tem um card no topo que leva para o detalhe do estabelecimento ligado a ele.

Tela de Notificações (`TelaNotificacoes.kt`). Acessada pelo sino na `TopAppBar`. Lista avisos em `LazyColumn`, diferencia visualmente lidos de não lidos, marca como lido ao toque e permite excluir. Ela

resolveu um problema concreto: o sino existia desde o Trabalho 1 e não fazia nada, o que o enunciado proíbe. Além disso, adicionar um estabelecimento ou um agendamento gera uma notificação nova, com o contador no sino subindo na hora.

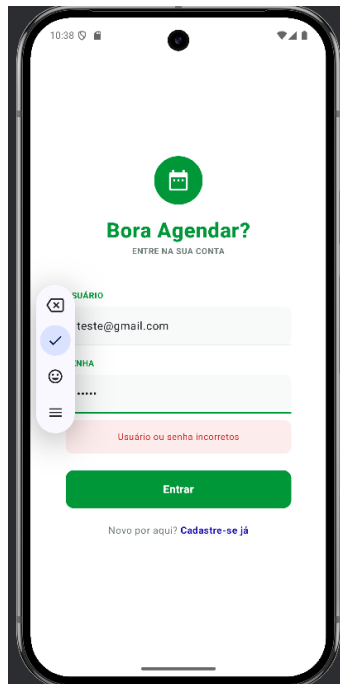


Figura 3. Tela de login recusando a entrada e exibindo o motivo após tentativa com senha incorreta.

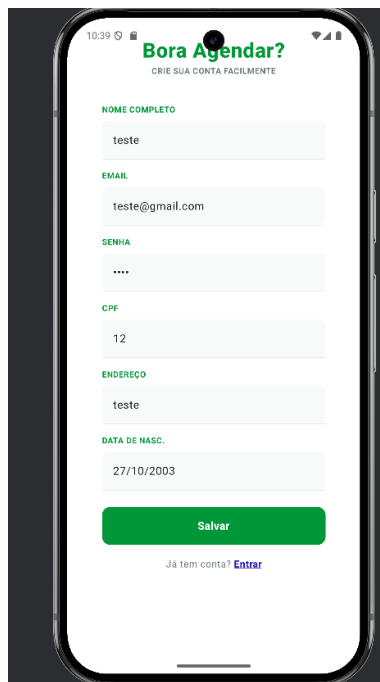


Figura 4. Tela de cadastro preenchida, com o campo de senha acrescentado no Trabalho 2.

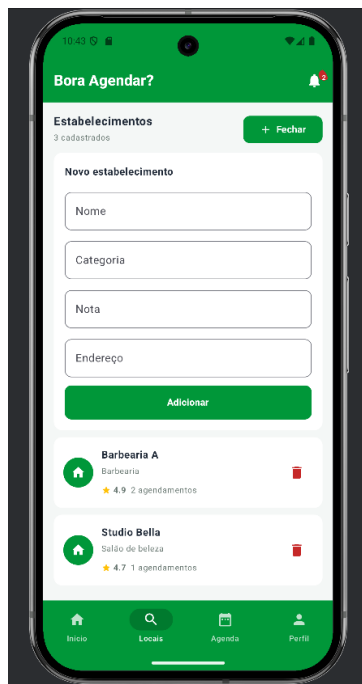


Figura 5. Aba Locais com o formulário de novo estabelecimento aberto e a lista logo abaixo.

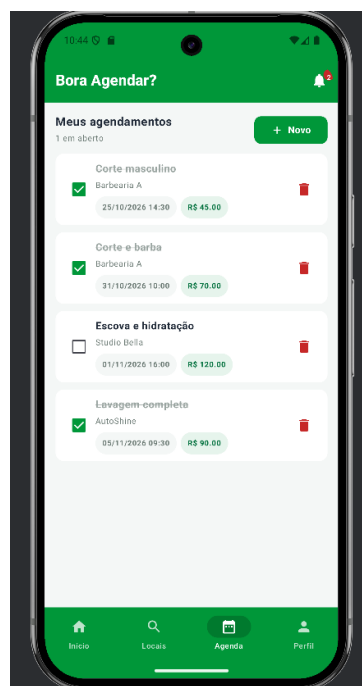


Figura 6. Aba Agenda, com itens marcados como concluídos pelo Checkbox e o preço em etiqueta.

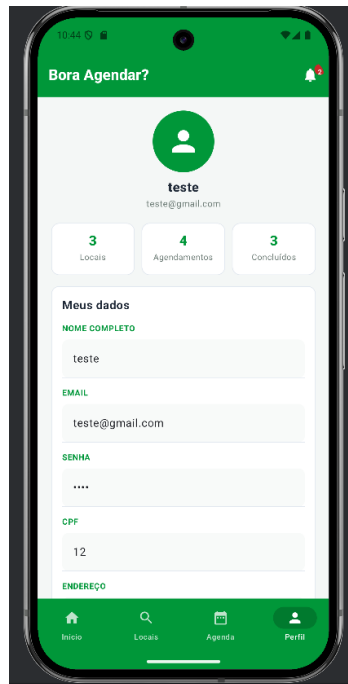


Figura 7. Aba Perfil, com os três contadores calculados a partir das listas e os dados do cadastro.

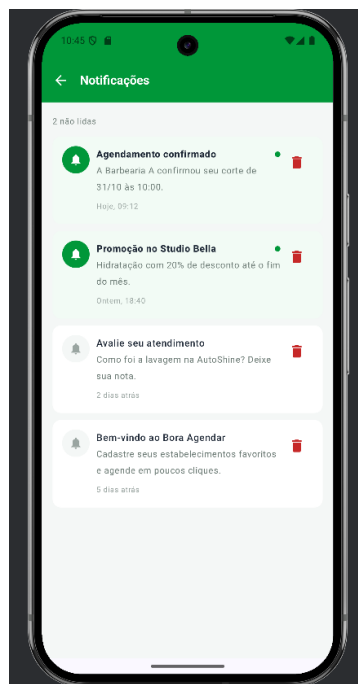


Figura 8. Tela de notificações, com os itens não lidos em destaque e o contador no sino.

3. Decisões de configuração e organização do código

Um arquivo por tela. No Trabalho 1 tudo ficava dentro da MainActivity.kt, que passou de quinhentas linhas e virou um problema para trabalhar em três pessoas, porque qualquer alteração dava conflito no Git. Separamos em dezessete arquivos, um por tela ou responsabilidade. A MainActivity.kt ficou com dezenove linhas e só chama AppNavigation().

Rotas em um objeto, nunca em texto solto. Todas as rotas são const val dentro de Rotas.kt. As duas rotas que levam argumento ganharam funções auxiliares:

```
const val DETALHE_ESTABELECIMENTO = "detalhe_estabelecimento/{id}"
```

```
fun detalheEstabelecimento(id: Int) = "detalhe_estabelecimento/$id"
```

A constante é o molde que o NavHost registra e a função monta a rota já com o id dentro. Fizemos assim porque escrever a rota na mão em cada navigate era onde mais errávamos digitação, e erro de rota em texto só aparece quando o aplicativo quebra, nunca na compilação.

Dois NavHost, não um. O NavHost externo, em AppNavigation.kt, controla login, cadastro, principal, notificações e os dois detalhes. O NavHost interno, dentro de MinhaTela.kt, controla apenas as quatro abas. Essa separação resolve uma questão visual: as abas precisam da barra inferior sempre visível, enquanto as telas de detalhe devem ocupar a tela inteira com o botão de voltar. Com um NavHost só, ou a barra inferior aparecia no login, ou sumia nas abas.

Lista no ViewModel, não na tela. As listas são mutableStateListOf dentro de MeuViewModel, criado uma única vez em AppNavigation com viewModel(). Como a mesma instância desce para todas as telas, o que é adicionado na aba Agenda aparece na aba Início sem nenhum código extra de sincronização. Se as listas ficassem dentro das telas, cada troca de aba recriaria a lista vazia.

Data classes com val e alteração por copy(). Testamos primeiro com var dentro das data classes e a tela não redesenhava ao marcar um agendamento como concluído. O mutableStateListOf observa mudanças na lista, não dentro dos objetos. A solução foi deixar os campos como val e trocar o item inteiro na posição:

```
val posicao = agendamentos.indexOfFirst { it.id == item.id }  
agendamentos[posicao] = item.copy(concluido = !item.concluido)
```

Estado de formulário fora do item do LazyColumn. Os remember dos formulários ficam no topo do composable da tela. Quando deixamos dentro do bloco item, o texto digitado sumia ao rolar a lista, porque o item sai de composição quando passa da área visível.

Componentes repetidos extraídos. CampoTexto, CaixaErro, CaixaSucesso, TituloSecao, Etiqueta e CaixaVazia ficaram em Componentes.kt. O bloco de configuração de cores do TextField apareceria quatorze vezes espalhado pelo projeto se não fizéssemos isso.

Versões travadas no version catalog. Usamos AGP 8.8.0 com compileSdk 35. Quando aceitamos a sugestão automática do Android Studio para adicionar as dependências de navegação, ele trouxe a navigation 2.9 e o lifecycle 2.11, que exigem AGP 9.1 e compileSdk 37, e o projeto parou de compilar com oito erros de AAR metadata. Travamos a navigation em 2.8.9 e o lifecycle em 2.8.7, que são as últimas versões compatíveis com o AGP que estamos usando.

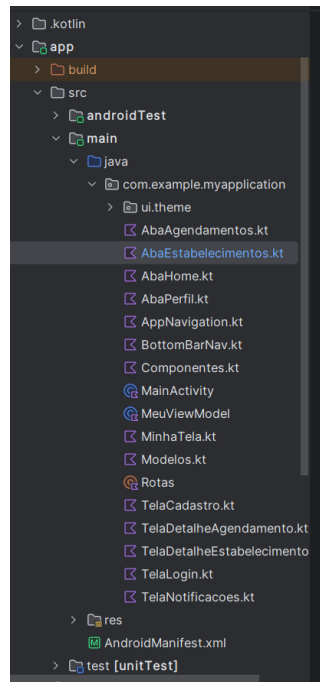


Figura 9. Painel Project do Android Studio, com os arquivos do pacote separados por responsabilidade.

4. A complexidade extra na tela de Detalhes

O enunciado pede que pelo menos uma das telas de detalhe faça mais do que reexibir os campos do item. Colocamos essa complexidade na Tela de Detalhe do Estabelecimento, e ela faz três coisas além de mostrar os dados.

Primeiro, combina as duas listas. A tela recebe apenas o id do estabelecimento. Com ele, filtra a lista de agendamentos buscando os que pertencem àquele local e exibe cada um deles em um card próprio:

```
fun agendamentosDoEstabelecimento(id: Int): List<Agendamento> {  
    return agendamentos.filter { it.idEstabelecimento == id }  
}
```

Segundo, mostra informação calculada. São quatro indicadores que não existem em nenhum campo das data classes e só aparecem porque são computados na hora: quantidade de agendamentos no local, quantos já foram concluídos, total gasto somando os preços e ticket médio dividindo o total pela quantidade.

```
val total = viewModel.totalGastoNoEstabelecimento(id)  
val media = if (agendamentos.isEmpty()) 0.0 else total / agendamentos.size
```

Terceiro, abre navegação secundária. Cada agendamento listado na tela é clicável e leva para o Detalhe do Agendamento, que por sua vez tem um card no topo que volta para o Detalhe do Estabelecimento. É possível circular entre os dois detalhes sem passar pelas listas.

Escolhemos essa combinação porque ela responde a uma pergunta que o usuário realmente faria dentro de um aplicativo de agendamento: quanto eu já gastei nessa barbearia e quais foram os serviços. Um detalhe que apenas repetisse nome, categoria e endereço não acrescentaria nada ao que a lista já mostra. Além disso, o total gasto só é possível porque as duas data classes estão ligadas pelo campo `idEstabelecimento`, o que exercita a relação entre elas em vez de tratá-las como listas independentes.

Um efeito dessa ligação apareceu durante os testes: ao remover um estabelecimento, os agendamentos dele ficavam órfãos e a lista mostrava Local removido. Resolvemos apagando os agendamentos junto com o estabelecimento.

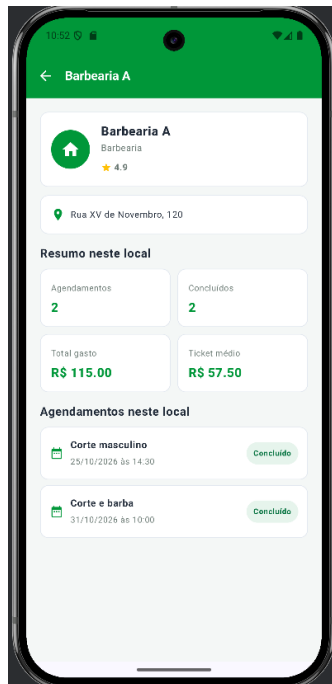


Figura 10. Detalhe da Barbearia A, com os quatro indicadores calculados e os agendamentos daquele local.

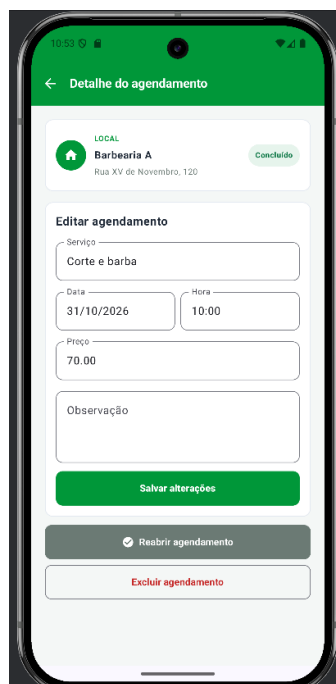


Figura 11. Detalhe do agendamento, com os campos em edição, incluindo a observação em múltiplas linhas.

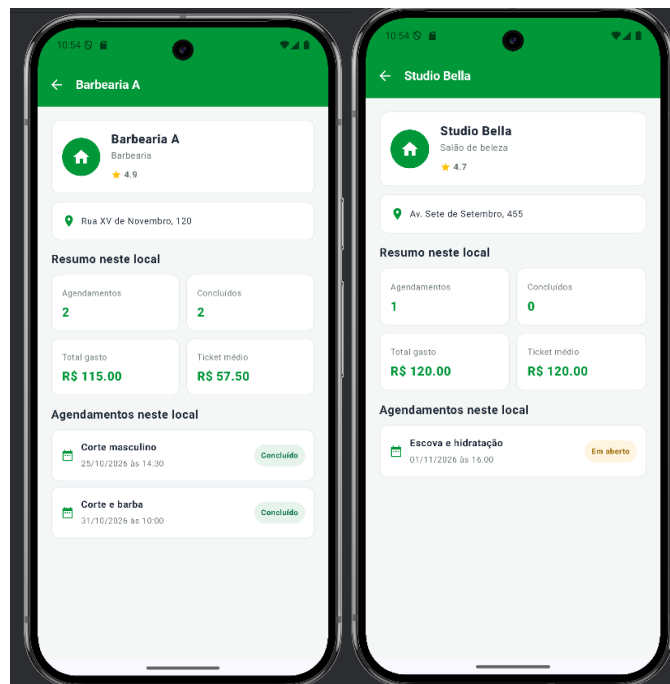


Figura 12. O mesmo detalhe aberto para dois locais diferentes. Os indicadores mudam conforme o item clicado.

5. Dificuldades e como resolvemos

Configuração do Gradle. Foi onde perdemos mais tempo, antes mesmo de escrever código. Ao adicionar a navegação pelo atalho do Android Studio, ele inseriu `navigation-runtime-android` e `navigation-compose-jvmstubs` no lugar de `navigation-compose`. O primeiro não tem o `NavHost` e o segundo é um pacote de stubs vazios para build de desktop, então nada funcionava e o erro não dizia isso. Resolvemos declarando a dependência correta na mão no `libs.versions.toml`. Depois disso veio o conflito de versões descrito na seção 3.

Projeto na pasta errada. Em uma das máquinas o projeto foi criado dentro de Documentos, que é sincronizado com o OneDrive, e o Gradle não conseguia montar o módulo, exibindo `Unable to find Gradle tasks to build`. Movemos para `AndroidStudioProjects` e o problema acabou. Também aprendemos que copiar arquivos `.kt` pelo Explorador de Arquivos com o projeto aberto não faz o Android Studio enxergá-los: é preciso usar `Reload from Disk` ou invalidar os caches.

Checkbox que não atualizava. Marcar um agendamento como concluído não mudava nada na tela. Levamos um tempo até entender que o `mutableStateListOf` observa a lista e não os campos internos dos objetos. A correção está descrita na seção 3.

Texto sumindo no formulário. O formulário de adicionar item perdia o que estava digitado quando rolávamos a lista. Descobrimos que era o `remember` declarado dentro do bloco `item` do `LazyColumn`, que é descartado quando o item sai da tela. Subimos os `remember` para o topo do `composable`.

Conteúdo embaixo das barras do sistema. Com `enableEdgeToEdge()` ativado, a barra inferior do aplicativo ficava escondida atrás da barra de navegação do Android e o título encostava na barra de status. Tentamos primeiro resolver na mão com `statusBarsPadding()` e `navigationBarsPadding()`. Depois que passamos a usar `Scaffold` com `TopAppBar` e `NavigationBar` do Material 3, o problema desapareceu, porque esses componentes já tratam as áreas do sistema sozinhos.

Empilhamento de histórico nas abas. No começo, trocar de aba empilhava destinos indefinidamente e o botão voltar do Android percorria todo esse histórico antes de sair da tela. Corrigimos adicionando `popUpTo`, `launchSingleTop` e `restoreState` na navegação da barra inferior.

6. Onde cada requisito foi atendido

Requisito do enunciado	Onde está
Mínimo de 7 telas navegáveis	10 telas, seção 2
NavHost central com objeto Rotas	<code>AppNavigation.kt</code> e <code>Rotas.kt</code>
NavigationBar funcionando	<code>BottomBarNav.kt</code> , quatro abas
Botões chamando <code>navigate</code> de verdade	Todas as telas, nenhum <code>onClick</code> vazio
TopAppBar com voltar funcional	Telas de detalhe e notificações, com <code>popBackStack()</code>
2 data classes	3 em <code>Modelos.kt</code> : <code>Estabelecimento</code> , <code>Agendamento</code> e <code>Notificacao</code>
2 telas de Lista com <code>LazyColumn</code> e <code>Card</code>	<code>AbaEstabelecimentos.kt</code> e <code>AbaAgendamentos.kt</code>
Lista reativa com <code>mutableStateListOf</code>	<code>MeuViewModel.kt</code>
Adicionar item pela interface	Formulário em cada uma das duas listas
Remover ou marcar item	Lixeira nas duas listas, <code>Checkbox</code> na de agendamentos
2 telas de Detalhes com o item correto	Argumento de rota com <code>NavType.IntType</code>
Complexidade extra no Detalhe	Seção 4 deste documento
Variedade de componentes	<code>TextField</code> , <code>OutlinedTextField</code> , campo numérico, campo de múltiplas linhas, <code>Checkbox</code> , <code>Card</code> , <code>Icon</code> , <code>IconButton</code> , <code>Button</code> , <code>OutlinedButton</code> , <code>Scaffold</code> , <code>Surface</code> , <code>LazyColumn</code> , <code>Row</code> , <code>Column</code> , <code>Box</code>